

CPSC 250

Operator Overloading

Edward Brash

1 Introduction

Earlier, we discussed overloading comparison operators such as:

`<, >, ==, !=`

using methods such as:

```
__lt__()
```

However, Python also allows us to overload arithmetic operators.

2 Why Arithmetic Overloading Matters

Suppose we define a `Time` class.

We might naturally want to compute:

```
time4 = time3 - time1
```

But what does subtraction mean for time objects?

Python does not know automatically.

Therefore, we must define the meaning ourselves.

3 The `__sub__()` Method

The subtraction operator:

`-`

is controlled by:

```
def __sub__(self, other):
```

Here:

- `self` is the left object,
- `other` is the right object.

4 Example: Time Difference

Suppose we define:

```
class Time:

    def __init__(self, hours, minutes):
        self.hours = hours
        self.minutes = minutes
```

We can implement subtraction:

```
def __sub__(self, other):

    time_diff = Time(0, 0)

    if self.minutes >= other.minutes:

        time_diff.minutes = self.minutes - other.minutes

        time_diff.hours = self.hours - other.hours

    else:

        time_diff.minutes = (
            self.minutes - other.minutes + 60
        )

        time_diff.hours = (
            self.hours - other.hours - 1
        )

    return time_diff
```

5 Understanding the Logic

Suppose:

```
time1 = Time(10, 45)
time2 = Time(8, 20)
```

Then:

```
time1 - time2
```

should represent:

$$10 : 45 - 8 : 20 = 2 : 25$$

If the minutes require borrowing, we borrow one hour and add 60 minutes.

6 Returning a New Object

Notice that the subtraction method creates a new `Time` object:

```
time_diff = Time(0, 0)
```

and returns it.

This is a common pattern in object-oriented programming.

7 Other Numerical Operator Methods

Python provides many special methods for arithmetic operations.

Method	Operator
<code>--add--</code>	<code>+</code>
<code>--sub--</code>	<code>-</code>
<code>--mul--</code>	<code>*</code>
<code>--truediv--</code>	<code>/</code>
<code>--floordiv--</code>	<code>//</code>
<code>--mod--</code>	<code>%</code>
<code>--pow--</code>	<code>**</code>

8 Logical Operator Methods

There are also methods associated with logical operations.

Method	Meaning
<code>--and--</code>	logical and
<code>--or--</code>	logical or

9 Conversion Methods

Python also allows customization of built-in conversion functions.

Method	Function
<code>--abs--</code>	<code>abs()</code>
<code>--int--</code>	<code>int()</code>
<code>--float--</code>	<code>float()</code>

10 Example: Integer Conversion

Suppose we define:

```
class Time:
    def __int__(self):
        return 60*self.hours + self.minutes
```

Then:

```
t = Time(2, 30)
print(int(t))
```

would produce:

150

because 2 hours and 30 minutes equals 150 total minutes.

11 The Big Idea

Operator overloading allows objects to behave naturally.

For example:

- time subtraction behaves like subtraction,
- object comparison behaves like comparison,
- printing behaves naturally,
- conversions behave naturally.

This makes programs easier to read and easier to use.

12 Key Takeaways

- Python allows arithmetic operator overloading.
- Special methods define the meaning of operators for custom classes.
- `__sub__` defines subtraction.
- Many arithmetic operators can be overloaded.
- Built-in conversion functions can also be customized.
- Overloading helps objects behave naturally and intuitively.